

MAE014 - Tóp. Ciência Dados B [Turmas MAP e EMA]

Preenchimento de Região em Matriz de Caracteres

Nome completo	DRE
Emanuel Cardoso	122033151
Alanna Figueiredo Simões	120053919
Felipe Rodrigues Schoeffer	123311188
Artur de Melo Brito	117224234

Data: 11/09/2026

Relatório de Bateria de Testes

Objetivo

Verificar o comportamento funcional dos dois algoritmos de preenchimento (recursivo e com pilha), a corretude em relação aos exemplos do enunciado, o comportamento em bordas abertas, o mecanismo de reprodução por passos e, por fim, demonstrar — com medições reais — que a versão final supera as versões ingênuas descritas no `CODE_REVIEW.md`.

Casos funcionais testados (`test_preenchimento.py`, `test_reprodutor.py`)

ID	Teste	Resultado esperado
T01	Figura pequena do enunciado, a partir de X	33 células preenchidas, sem repetição
T02	Célula isolada (2,3), cercada por paredes	Permanece 1 (não alcançável por 4-vizinhança)
T03	Borda da figura	Permanece intacta (não é sobrescrita)
T04	Figura com borda aberta (buraco proposital em (2,4))	Vaza: 158 de 192 células preenchidas
T05	Labirinto (exemplo grande do enunciado), a partir de X	502 células preenchidas, X vira 0
T06	Posição inicial fora dos limites	Nenhuma alteração, lista de eventos vazia
T07	Posição inicial no fundo externo (fora da figura)	Preenche normalmente (célula "alvo" válida)
T08	Posição inicial sobre parede real da figura (0)	Nenhuma alteração, lista de eventos vazia
T09	Posição inicial sobre célula já preenchida	Nenhuma alteração, lista de eventos vazia
T10	Recursivo vs. pilha, nos três exemplos	Resultado final idêntico nos dois algoritmos
T11	Reprodutor, $P = 0$	Aplica tudo de uma vez, nenhuma pausa, um único print final
T12	Reprodutor, $P > 0$	Pausa a cada bloco de P eventos, aguarda ENTER, título correto a cada bloco
T13	Reprodutor não altera a matriz original	A matriz passada como referência permanece intocada
T14	Entrada de velocidade (modo gráfico), valores inválidos e válidos	Repete o pedido até receber um número > 0

Execução:

```
cd classe_pilha_preenchimento
python -m pytest -v test_preenchimento.py test_reprodutor.py test_entrada_usuario.py
```

Resultado obtido nesta preparação: **26 testes aprovados** (2 algoritmos × 3 casos parametrizados + 1 teste de agregação + 4 testes de reprodutor + 3 testes de entrada de velocidade).

Testes de estresse (test_estresse.py) — versão final vs. versão ingênua

Execução:

```
cd classe_pilha_preenchimento
python -m pytest -v test_estresse.py
```

Resultado obtido nesta preparação: **6 testes aprovados**. Números medidos durante a preparação deste material (ambiente: container Linux, CPython 3.14; os valores absolutos variam por máquina, mas a relação entre as versões se manteve estável em execuções repetidas):

Preenchimento com pilha — uma pilha codificada vs. duas pilhas paralelas

Sala vazia de 100×100 (9.604 células internas), medição real desta preparação:

Versão	Operações de empilha	Capacidade da pilha
Ingênua (2 pilhas, push incondicional)	38.417	40.000 (estimativa ×4, ~4% de folga)
Final (1 pilha, push só quando válido)	9.604	9.604 (exata)

Redução de **4×** nas operações de pilha, com o mesmo resultado final.

Sala vazia de 200×200, comparação de tempo (menor tempo em 5 execuções):

Versão	Tempo
Ingênua	0,0665 s
Final	0,0230 s

Aceleração de **~2,9×** no tempo de execução.

Preenchimento recursivo — listas de caracteres vs. strings imutáveis

300 salas de 25×25 (custo de escrita isolado do tamanho da recursão), menor tempo em 5 execuções:

Versão	Tempo
Ingênua (lista de strings)	0,0659 s
Final (lista de listas de caracteres)	0,0506 s

Aceleração de **~1,3×** — mais modesta que no caso da pilha, porque nos tamanhos testados o custo de reconstruir a string ainda é pequeno perto do

overhead de cada chamada recursiva do Python. Ainda assim, o ganho é real e cresce com a largura da matriz.

Recursão sem proteção vs. recursão com thread dedicada

Cenário	Versão sem proteção	Versão final (thread + limite ajustado)
Sala vazia 60×60 (3.600 células)	RecursionError	OK
Sala vazia 250×250 (62.500 células)	não testado (já falha bem antes)	OK

Este é o achado mais decisivo dos testes de estresse: a versão recursiva ingênua **não é utilizável** para regiões de tamanho realista sem a correção — não se trata apenas de desempenho, e sim de a rotina terminar ou não.

Conclusão

A bateria cobre o fluxo funcional descrito no enunciado (preenchimento fechado, célula isolada, borda aberta, os três exemplos fornecidos) e, com os testes de estresse, evidencia numericamente que a versão final supera a versão ingênua gerada inicialmente pela IA nos três eixos pedidos: corretude (capacidade exata, sem vazamento indevido), desempenho (menos operações e menos tempo) e robustez (não estoura em regiões grandes). Para a entrega final, recomenda-se que o grupo repita `python -m pytest -v` no próprio ambiente e anexe a saída real do terminal.